

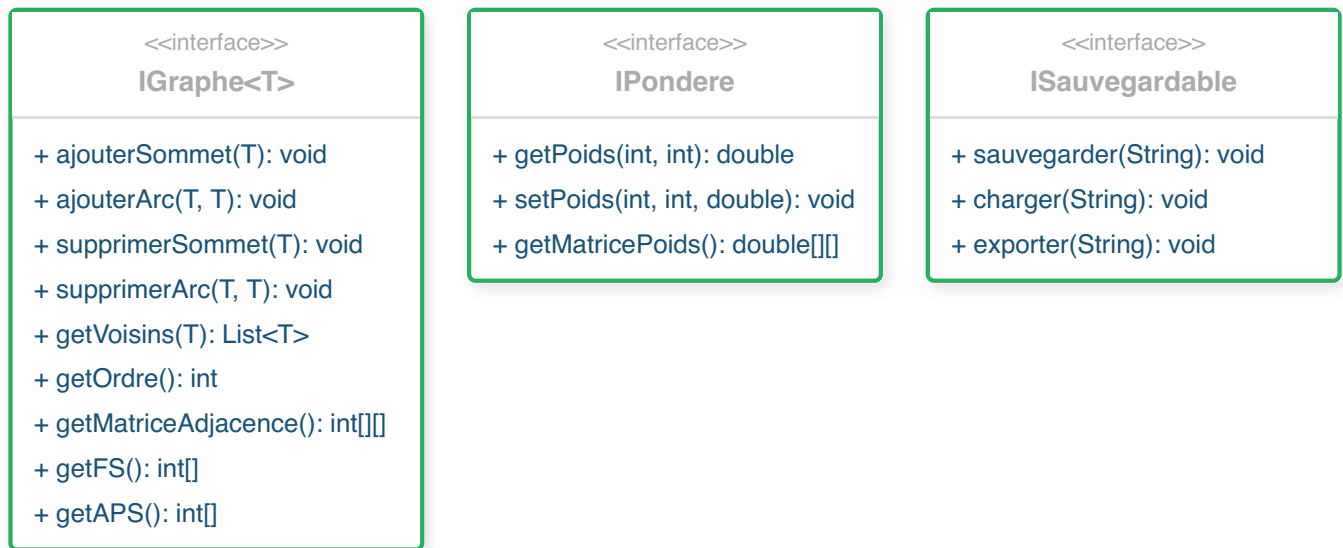
Architecture UML v2 — Projet Algorithmes de Graphes (Java)

Correction complète — Les 4 types de graphes + Interfaces + Généricité <T>

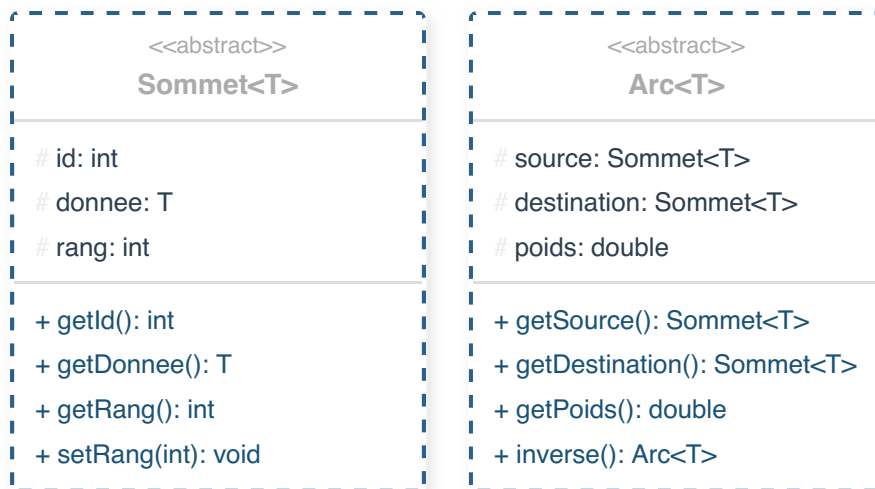
 Interface  Classe abstraite  Classe concrète (graphe)  Classe concrète (algo)

 Classe utilitaire  —  extends  -  implements  —  composition

Couche 1 — Interfaces



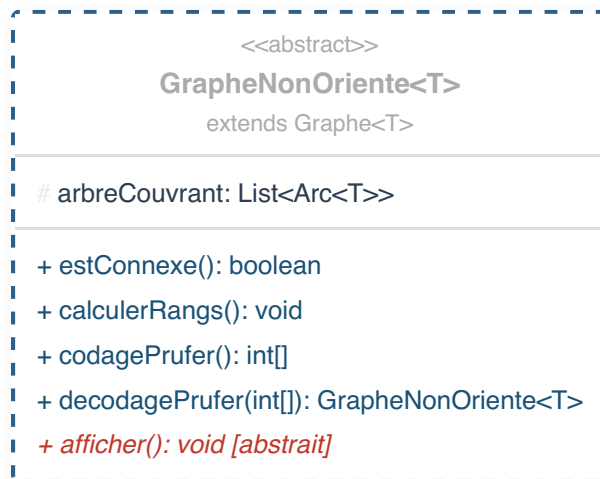
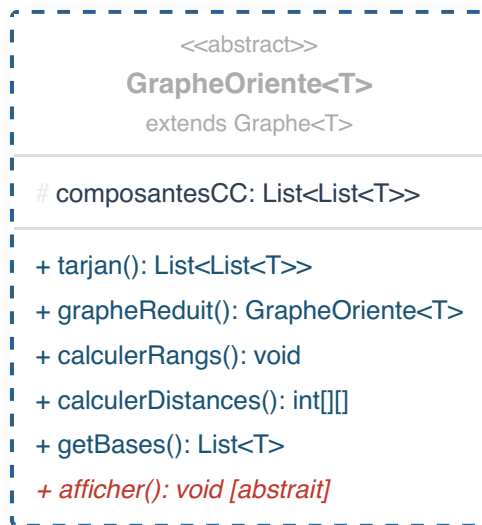
Couche 2 — Entités de base



Couche 3 — Classe abstraite Graphe<T>



Couche 4 — Les 2 grandes familles



Couche 5 — Les 4 classes concrètes finales

GrapheOrienteSimple<T>

extends GrapheOriente<T>

(graphe orienté non pondéré)

+ afficher(): void
+ toString(): String

◆ Utilisé par : Tarjan, Rangs, Distances, Ordonnement

GrapheOrientePondere<T>

extends GrapheOriente<T>

implements IPondere

(graphe orienté pondéré)

- matricePoids: double[][]

+ getPoids(int,int): double
+ setPoids(int,int,double): void
+ dijkstra(T,T): List<T>
+ dantzig(): double[][]
+ verifierConditions(): boolean
+ afficher(): void

◆ Utilisé par : Dijkstra, Dantzig

GrapheNonOrienteSimple<T>

extends GrapheNonOriente<T>

(graphe non orienté non pondéré)

+ afficher(): void
+ toString(): String
+ estArbre(): boolean

◆ Utilisé par : Codage de Prüfer

GrapheNonOrientePondere<T>

extends GrapheNonOriente<T>

implements IPondere

(graphe non orienté pondéré)

- matricePoids: double[][]

+ getPoids(int,int): double
+ setPoids(int,int,double): void
+ kruskal(): List<Arc<T>>
+ afficher(): void

◆ Utilisé par : Kruskal ✓ (manquait avant !)



Couche 6 — Classes utilitaires



Table des relations

Classe	Relation	Avec	Justification
Graphe<T>	implements	IGraphe<T>, ISauvegardable	Contrat commun à tous les graphes
Graphe<T>	composition	List<Sommet<T>>, List<Arc<T>>	Un graphe est composé de sommets et arcs
GrapheOriente<T>	extends	Graphe<T>	Spécialise le graphe avec

			orientation
GrapheNonOriente<T>	extends	Graphe<T>	Spécialise le graphe sans orientation
GrapheOrienteSimple<T>	extends	GrapheOriente<T>	Graphe orienté non pondéré → Tarjan, Rangs
GrapheOrientePondere<T>	extends + implements	GrapheOriente<T> + IPondere	Graphe orienté pondéré → Dijkstra, Dantzig
GrapheNonOrienteSimple<T>	extends	GrapheNonOriente<T>	Graphe non orienté non pondéré → Prüfer
GrapheNonOrientePondere<T>	extends + implements	GrapheNonOriente<T> + IPondere	✅ Graphe non orienté pondéré → Kruskal
Ordonnancement	dépendance	GrapheOrienteSimple<Tache>	Utilise le graphe orienté pour les tâches
GestionnaireGraphe	composition	Graphe<?> (pattern Singleton)	Gère le graphe courant + historique

Quel algorithme dans quelle classe ?

GrapheOrienteSimple<T>

- ✅ Tarjan (composantes fortement connexes)
- ✅ Graphe réduit + Bases
- ✅ Calcul des rangs
- ✅ Calcul des distances
- ✅ Ordonnancement (via classe dédiée)

GrapheOrientePondere<T>

- ✅ Dijkstra (+ vérification : pas de poids négatif)
- ✅ Dantzig (+ vérification conditions)

GrapheNonOrienteSimple<T>

- ✅ Codage de Prüfer

GrapheNonOrientePondere<T> ← manquait !

- ✅ Kruskal (arbre couvrant minimal)

- ✓ Decodage de Pruter
- ✓ Vérification si c'est un arbre

- ✓ Kruskal (arbre couvrant minimal)
- ✓ Vérification connexité (obligatoire pour Kruskal)



Pourquoi IPondere est une interface et non une classe mère ?

En Java, on ne peut hériter que d'une seule classe (*single inheritance*). Si "Pondéré" était une classe,

`GrapheOrientePondere` ne pourrait pas hériter à la fois de `GrapheOriente` ET de `Pondere`.

L'interface résout ce problème élégamment — c'est exactement le cas d'usage pour lequel les interfaces ont été créées en Java.